

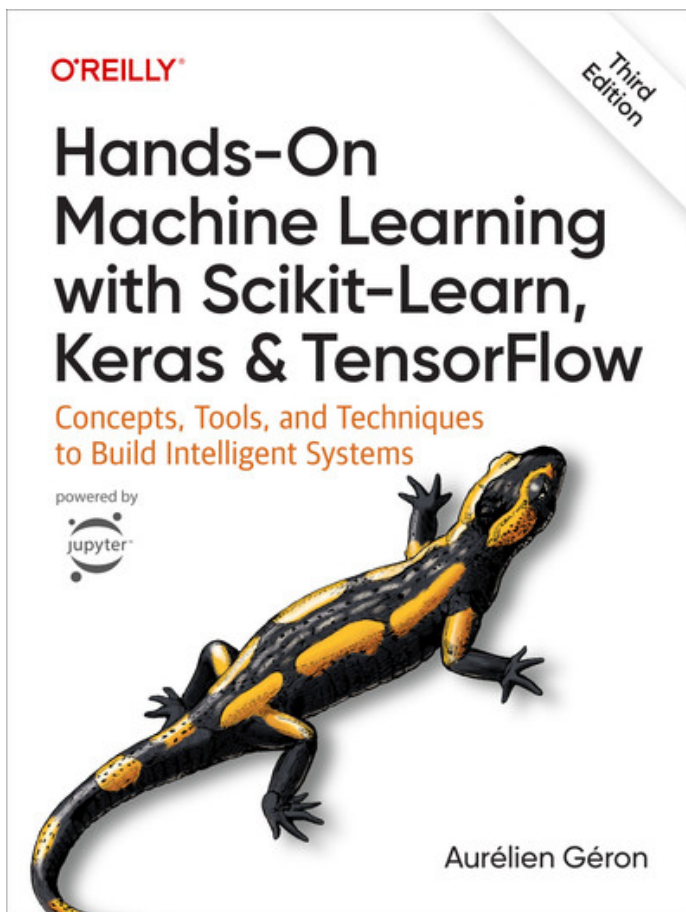
Árboles de Decisión

Técnicas de aprendizaje de máquina

Oscar Bustos

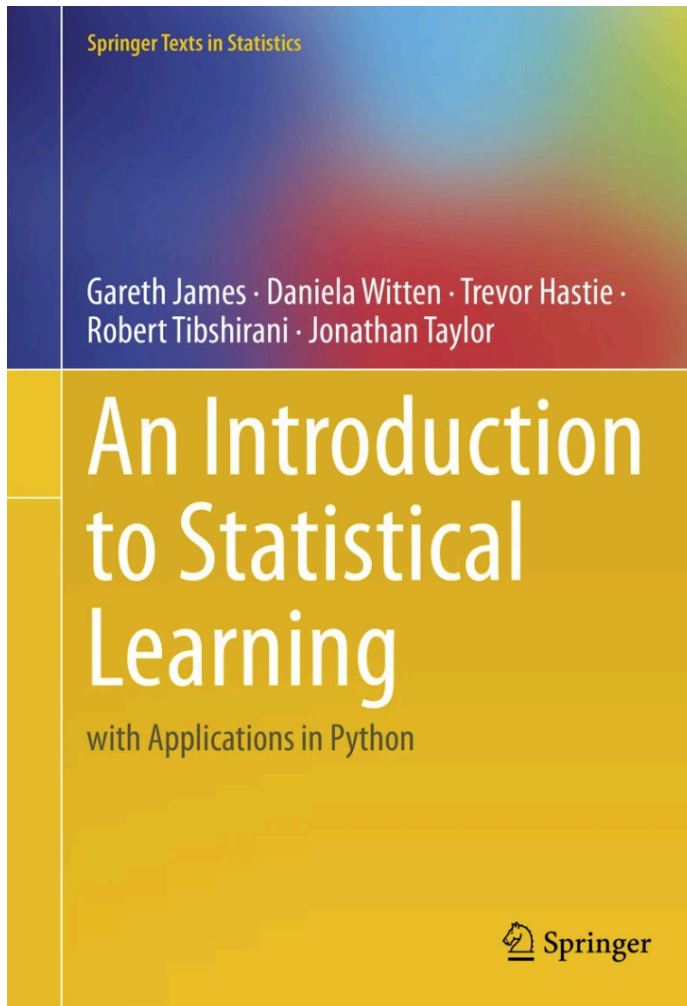
Intuición Matemática

Capítulos sobre Árboles

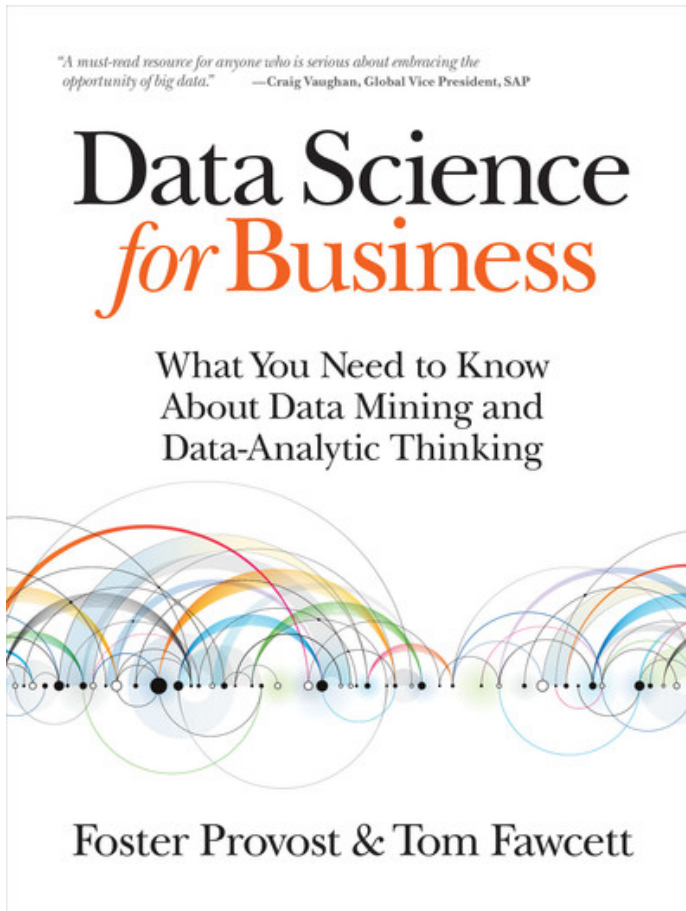


\ Capítulo 6

Capítulos sobre Árboles



\ Capítulo 8

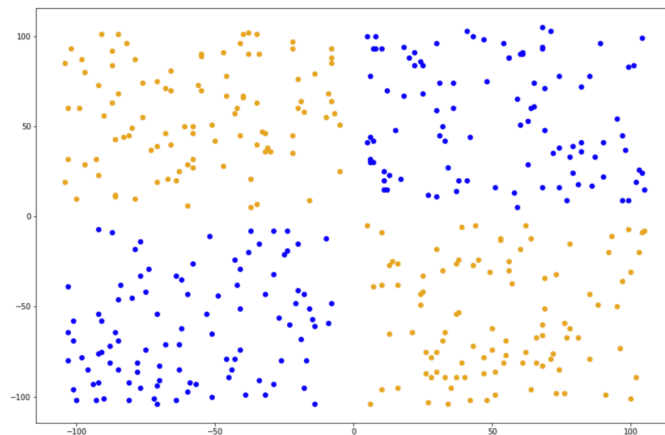


\ Capítulo 3

Problema XOR

¿Cómo construyo un modelo lineal para separar perfectamente los puntos de la siguiente gráfica?

% Espacio opcional entre el texto y la imagen



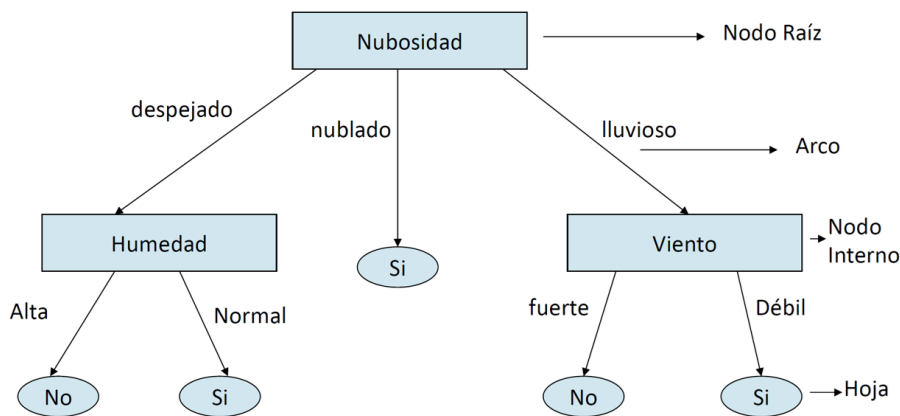
Enfoque Árboles de Decisión

- Conjunto de **condiciones** organizadas en una estructura **jerárquica**, de tal manera que la decisión final a tomar se pueda determinar siguiendo las condiciones que se cumplen desde la raíz del árbol hasta alguna de sus hojas.
- Cada *nodo interior* contiene una pregunta sobre un atributo concreto (con un hijo por cada posible respuesta) y cada *nodo hoja* se refiere a una decisión (clasificación).

Estructura de Árbol

¿Va a llover hoy?

% Espacio opcional entre el texto y la imagen



Ejemplo de Clasificación

- ¿Podemos predecir si un usuario cae en mora o no?
- Modelo: $Mora \approx f(\text{Balance, Edad, Tiene Empleo})$

Attributes				Target attribute
Name	Balance	Age	Employed	Write-off
Mike	\$200,000	42	no	yes
Mary	\$35,000	33	yes	no
Claudio	\$115,000	40	no	no
Robert	\$29,000	23	yes	yes
Dora	\$72,000	31	no	no

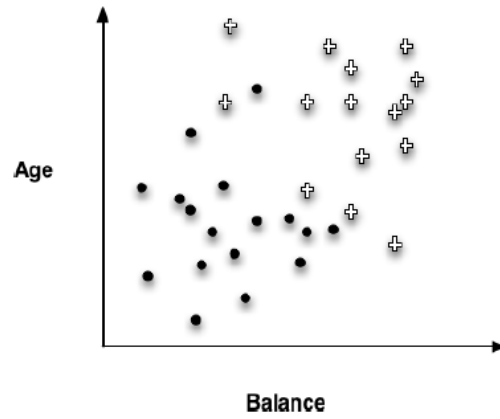
This is one row (example).
 Feature vector is: **<Claudio,115000,40,no>**
 Class label (value of Target attribute) is **no**

\end{center}

Ejemplo de Clasificación

- ¿Podemos predecir si un usuario cae en mora o no?

•

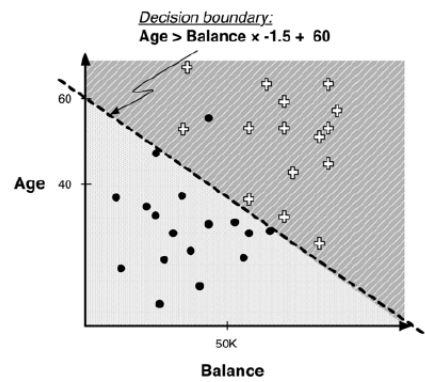


Modelo: $Mora \approx f(\text{Balance}, \text{Edad})$

Ejemplo de Clasificación

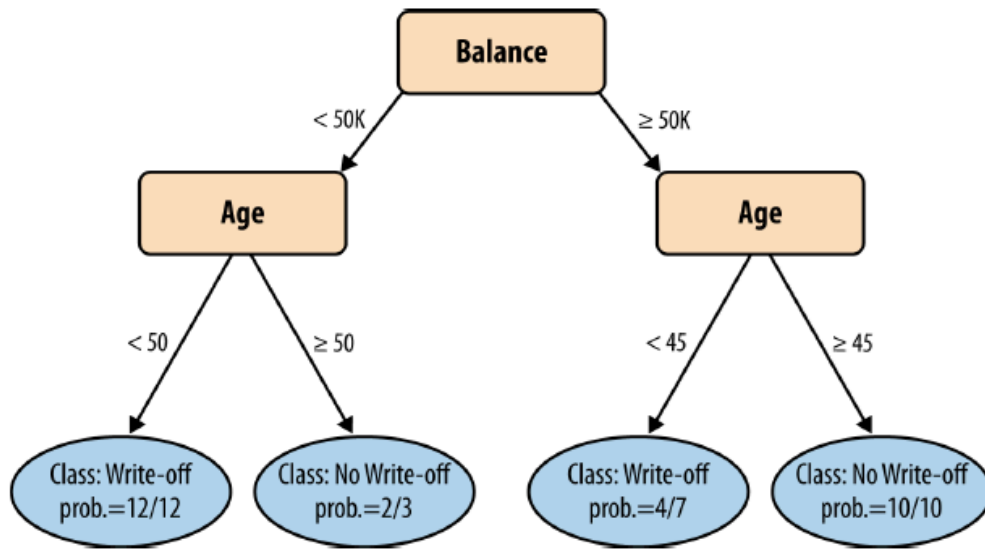
- Solución: Regresor Lineal

- Región de Decisión: $\$ \quad P(\quad = \quad 1) =$



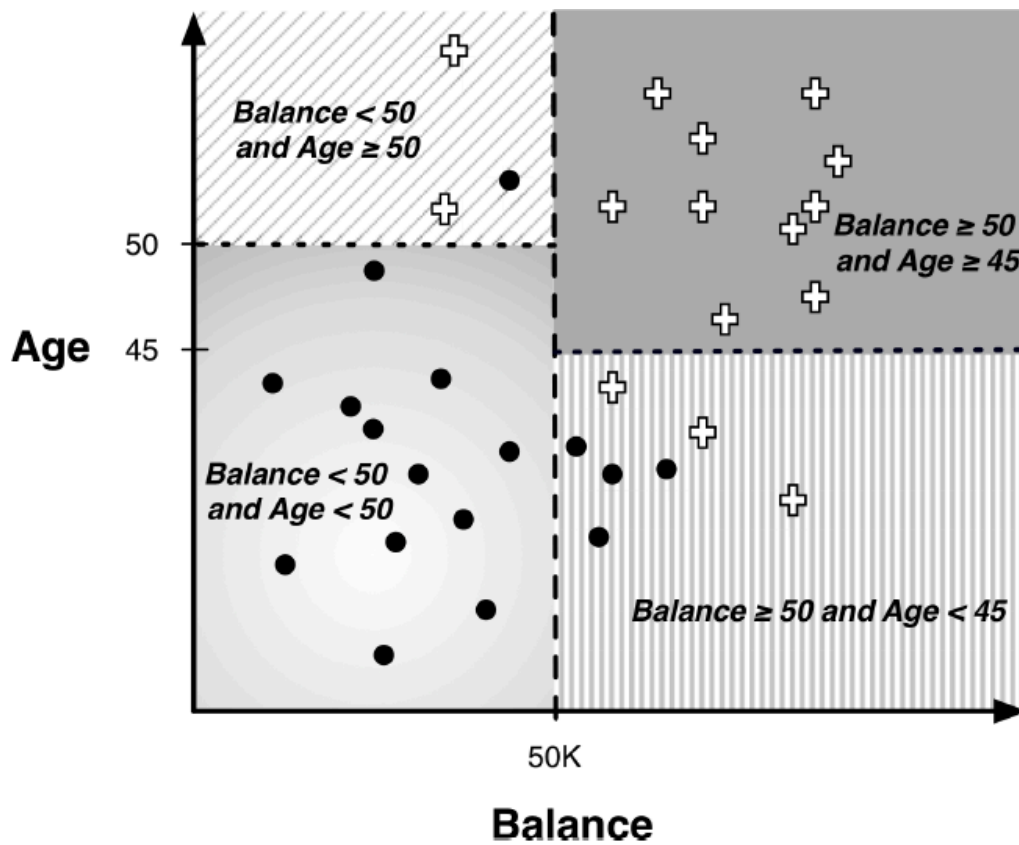
\$

Ejemplo de Clasificación



\ Árbol

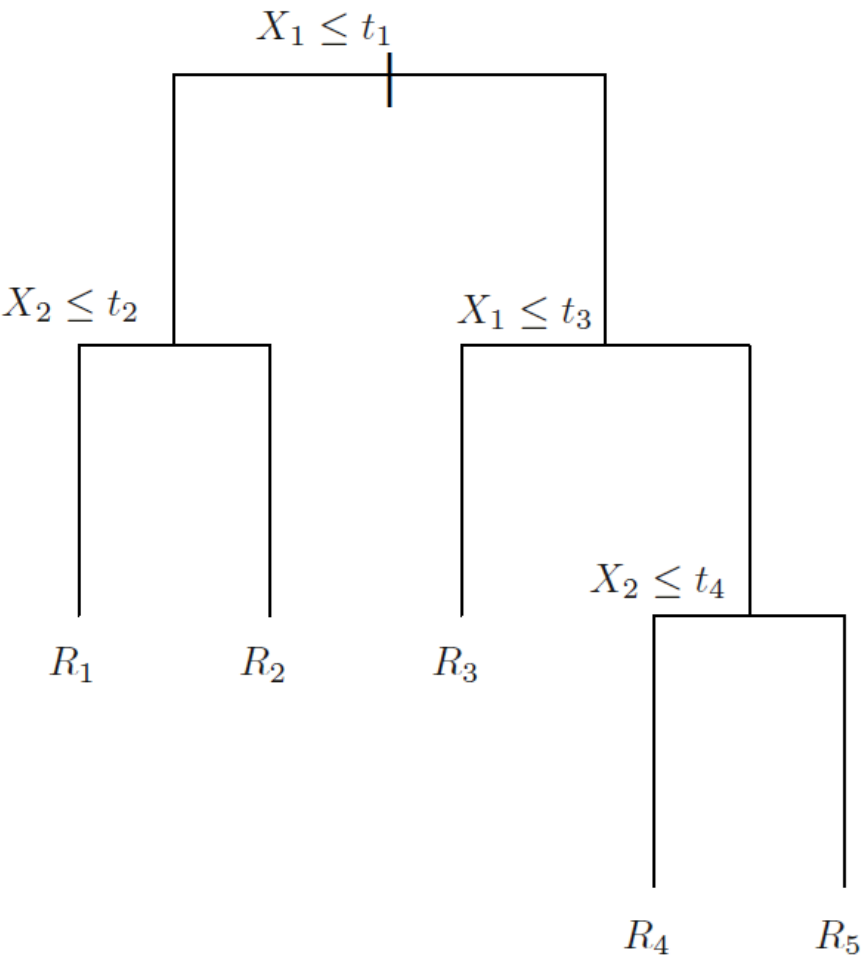
de Decisión



\ Región

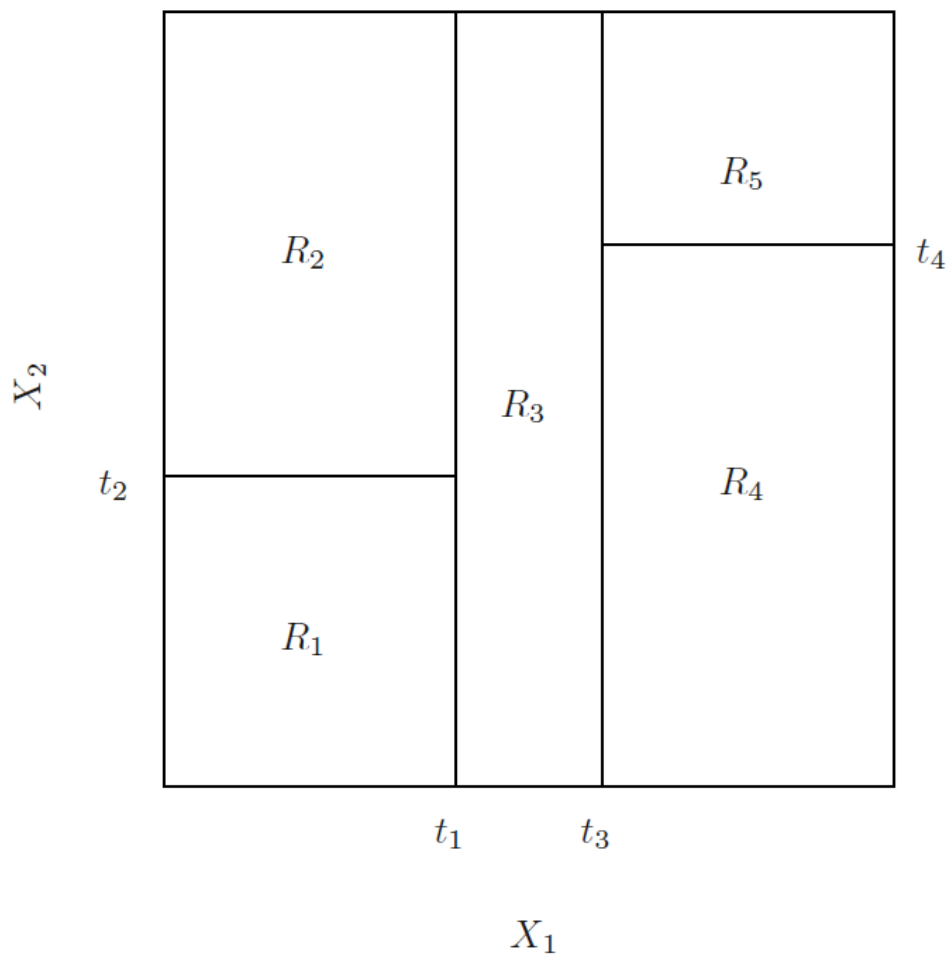
de Decisión

Ejemplo de Regresión



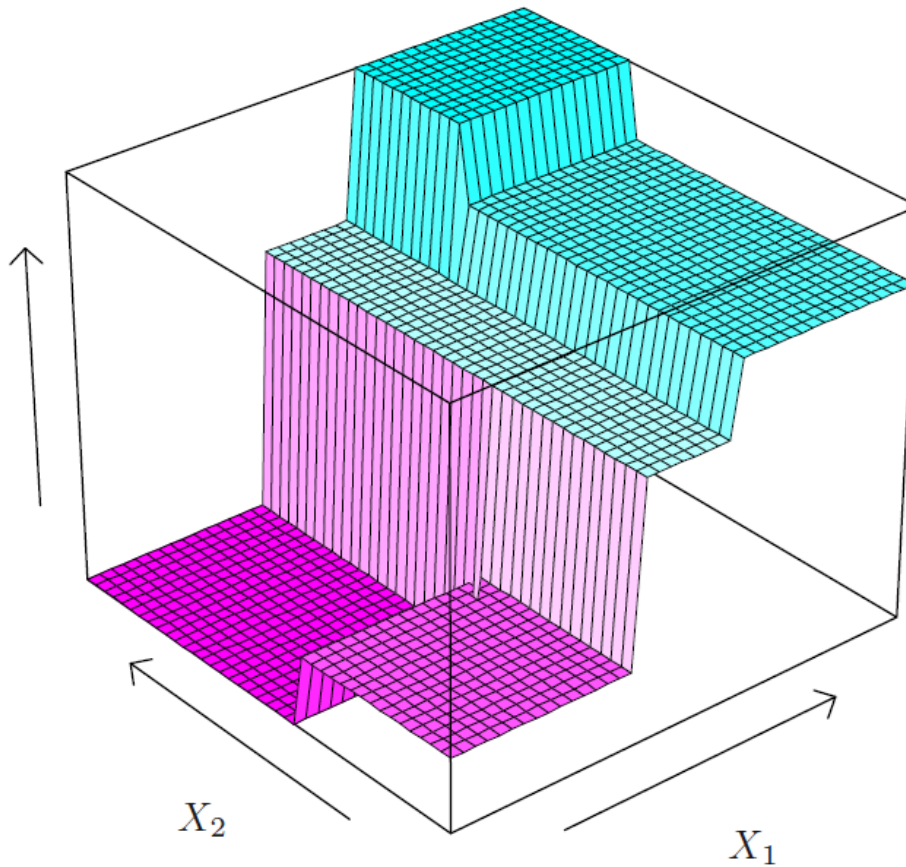
de Regresión

\ Árbol



giones de Regresión

\ Re-



\ Superfi-

cie de Regresión

Algoritmos

Construcción del Árbol de Decisión

- **Entrenamiento:**

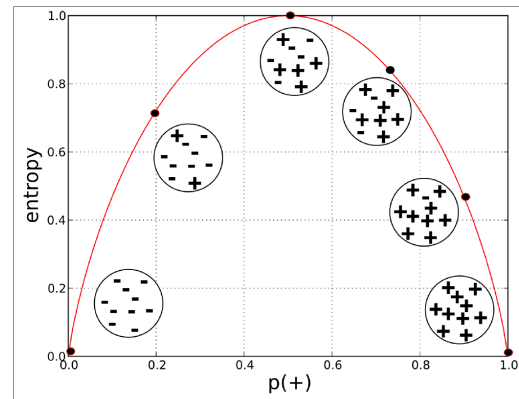
- Construye el árbol con datos de entrenamiento.
- **Objetivo:** Maximizar separación de clases (clasificación) o reducir varianza (regresión) al dividir nodos.
- **Criterios de División:**
 - *Clasificación:* Entropía, Gini, Ganancia de Información.
 - *Regresión:* RSS (Reducción Suma de Cuadrados Residuales).

- **Prueba (Predicción):**

- Aplica el árbol a nuevos datos para predecir.
- **Funcionamiento:** Recorre el árbol desde la raíz, evaluando características en cada nodo hasta llegar a una hoja (predicción).

Ganancia de Información

- Entropía en un Nodo
-



$$entropy = -p_1 \log(p_1) - p_2 \log(p_2) - \dots$$

Escenario del Ejemplo

Escenario: Nodo *S* con 10 individuos.

Imagina que estamos analizando un nodo en un árbol de decisión, al que llamamos “Nodo S”. Este nodo representa un conjunto de datos con 10 individuos.

Clasificación: Dentro de estos 10 individuos, tenemos dos categorías importantes:

- **3 individuos en mora** (nuestra clase positiva, por ejemplo)
- **7 individuos sin mora** (nuestra clase negativa)

Cálculo de Probabilidades

i Cálculo de Probabilidades

Para calcular la entropía, primero necesitamos las probabilidades de cada clase en el Nodo S.

- Probabilidad de “no mora”: [$p() = 0.7$]
- Probabilidad de “mora”: [$p() = 0.3$]

Fórmula de Entropía

i Fórmula de Entropía (para dos clases)

La fórmula general para calcular la entropía en un sistema con dos clases (como “mora” y “no mora”) es:

$$[(S) = - p() _2(p()) - p() _2(p())]$$

Donde: - S representa el conjunto de datos (nuestro Nodo S) - $p(\text{mora})$ es la probabilidad de la clase “mora” - $p(\text{no-mora})$ es la probabilidad de la clase “no mora” - $\log_2$ es el logaritmo en base 2

Cálculo de Entropía (Nodo S)

i Sustituyendo valores en la fórmula

Ahora, sustituimos las probabilidades que calculamos en la fórmula de la entropía:

$$[(S) = - _2 () - _2 ()]$$

i Resultado Aproximado

Calculando esta expresión, obtenemos:

$$[(S)]$$

Interpretación del Resultado

Interpretación: Este valor de entropía (aproximadamente 0.88 bits) representa una medida del desorden o la impureza en el nodo S con respecto a la clasificación de “mora” y “no mora”.

En términos más simples, nos dice qué tan mezcladas están las clases en este nodo. Una entropía más alta indicaría más desorden (más mezcla de clases), mientras que una entropía más baja indicaría menos desorden (las clases están más separadas).

Este valor de entropía será crucial para calcular la Ganancia de Información al dividir este nodo en un árbol de decisión.

Ganancia de Información: Midiendo la Mejora

Introducción a la Ganancia de Información (IG)

Hemos calculado la entropía, que mide la impureza de un nodo. Ahora, necesitamos una métrica para evaluar **cuánto “ganamos”** al dividir un nodo padre en nodos hijos. Esta métrica es la **Ganancia de Información**.

- **Objetivo de la Ganancia de Información:**
 - Medir la **reducción en entropía** después de dividir un nodo.

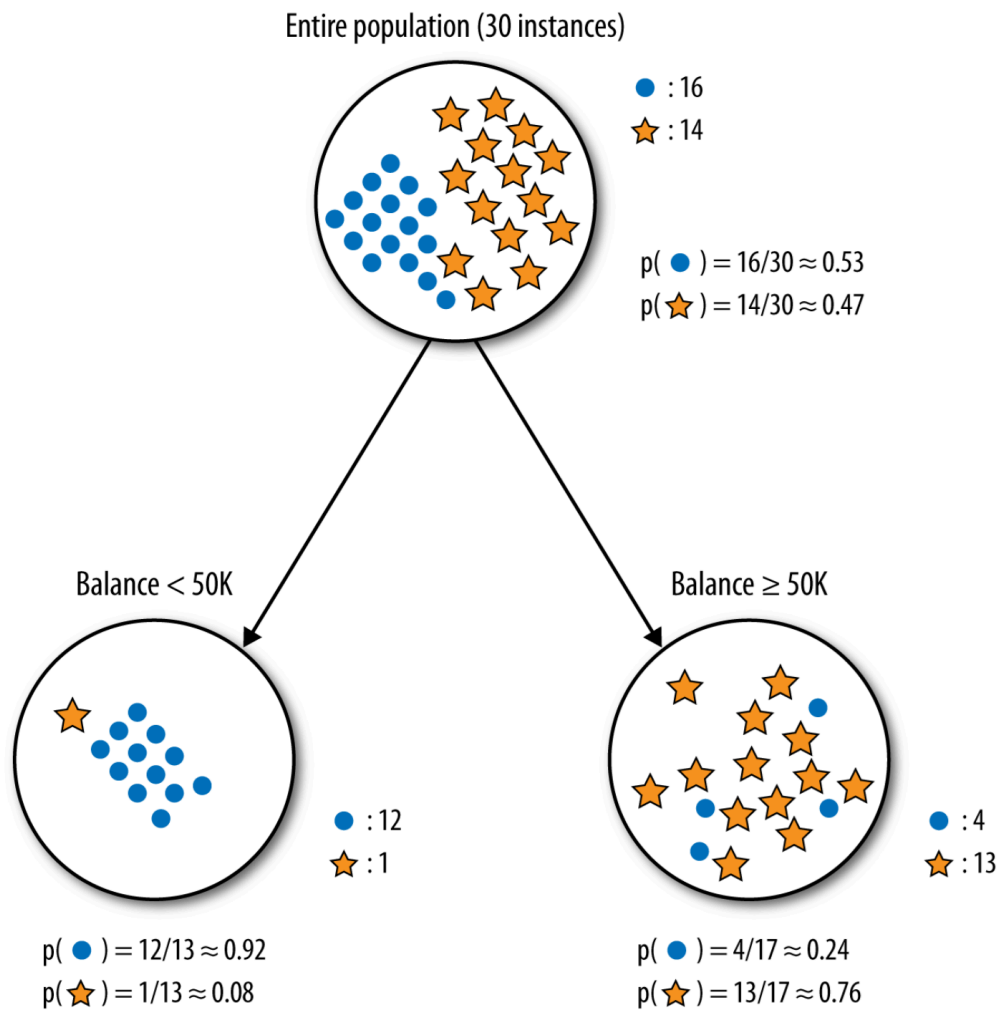
- Cuantificar la **efectividad de una división** para clasificar los datos.
- **Mayor IG $\implies$ Mejor División**: Divisiones con mayor Ganancia de Información son preferibles en la construcción de árboles de decisión.

Ganancia de Información: Midiendo la Mejora

La fórmula para calcular la Ganancia de Información de un nodo padre al dividirlo en nodos hijos es: $IG(\text{padre}, \text{hijos}) = \text{entropy}(\text{padre}) - \sum_{i=1}^n p(\text{hijo}_i) \times \text{entropy}(\text{hijo}_i)$ Donde: - $IG(\text{padre}, \text{hijos})$: Ganancia de Información al dividir el nodo “padre” en nodos “hijos”. - $\text{entropy}(\text{padre})$: Entropía del nodo padre **antes** de la división. - n : Número de nodos hijos creados por la división. - $p(\text{hijo}_i)$: Proporción de individuos que se dirigen al nodo hijo i . (Probabilidad de llegar al hijo i). - $\text{entropy}(\text{hijo}_i)$: Entropía del nodo hijo i **después** de la división. - $\sum_{i=1}^n p(\text{hijo}_i) \times \text{entropy}(\text{hijo}_i)$: Suma ponderada de las entropías de los nodos hijos.

Ganancia de Información

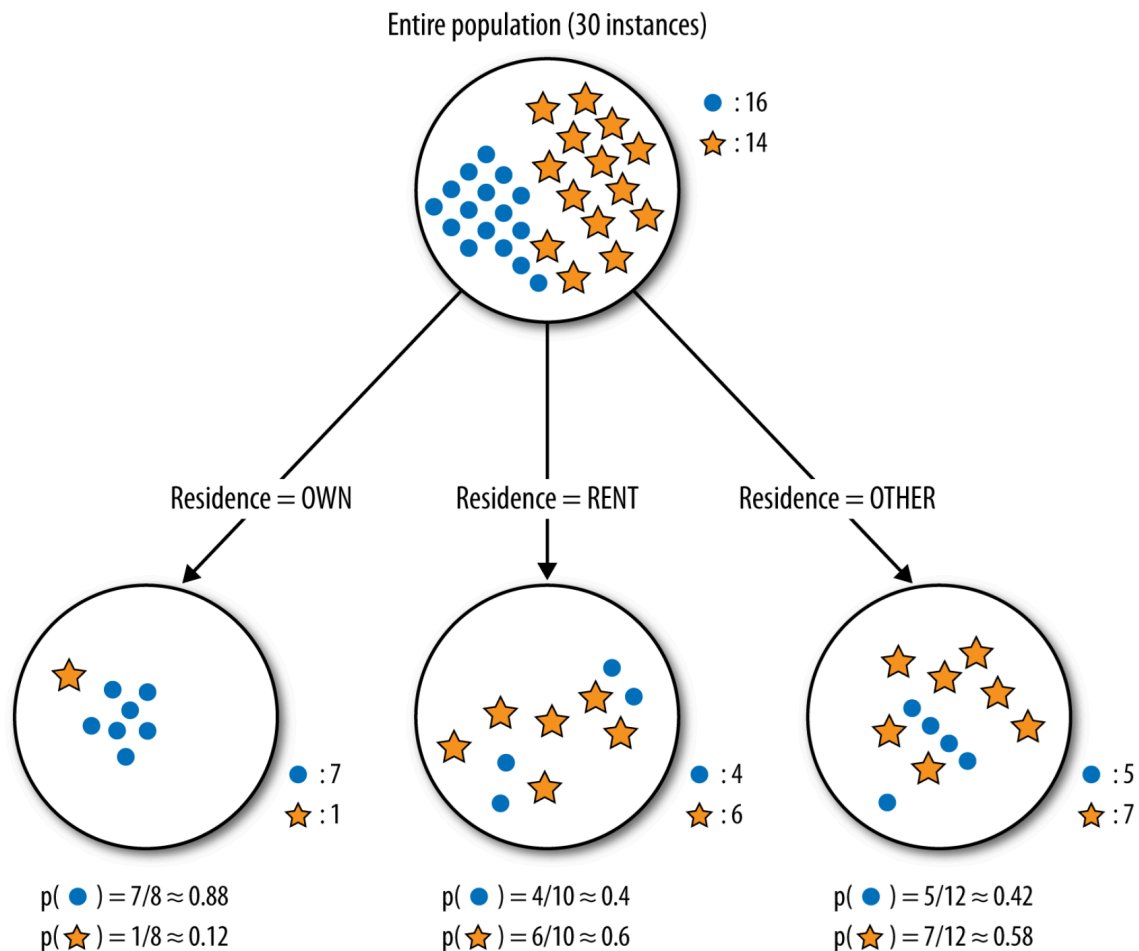
$IG = \text{entropy}(\text{parent}) - p(\text{Balance} < 50K) \times \text{entropy}(\text{Balance} < 50K) + p(\text{Balance} \geq 50K) \times \text{entropy}(\text{Balance} \geq 50K) \setminus IG \approx 0.99 - 0.43 \times 0.39 + 0.57 \times 0.79 \setminus IG \approx 0.37$



\ Partición usando el Balance

Ganancia de Información

$entropy(parent) \approx 0.99$ $entropy(Residence = OWN) \approx 0.54$ $entropy(Residence = RENT) \approx 0.97$ $entropy(Residence = OTHER) \approx 0.98$ $IG \approx 0.13$



\ Partición usando la Residencia

Proceso de Poda

- Eliminar hojas, ramas o subárboles para mejorar el desempeño del árbol
- El árbol sin podar clasificará correctamente los registros del set de entrenamiento pero clasificará incorrectamente algunos registros del set de pruebas
- Compromiso entre la complejidad del árbol y el error de clasificación

Comparativa: Árboles vs. Regresión

Decisión clave: ¿Árboles de Decisión o Regresión Lineal/Logística?

Depende de: - La naturaleza de los datos. - El tipo de problema. - La necesidad de interpretabilidad.
- La eficiencia requerida.

Exploremos las ventajas y desventajas de cada uno en varios slides.

Ventajas de los Árboles: No Linealidad e Interpretación

i Ventajas Clave

- **Relaciones No Lineales:** Capturan patrones complejos sin transformaciones manuales.
- **Interpretación Visual:** Estructura clara y fácil de visualizar para explicar decisiones.

Ideal cuando: - Sospechas relaciones no lineales en los datos. - La interpretabilidad del modelo es prioritaria.

Ventajas de los Árboles: Datos Mixtos y Outliers

i Otras Ventajas

- **Datos Categóricos y Numéricos:** Manejan ambos tipos de datos directamente.
- **Robustos a Outliers:** Menos afectados por valores atípicos.
- **Selección de Variables Implícita:** Identifican variables relevantes automáticamente.

Desventajas de los Árboles: Overfitting e Inestabilidad

i Desventajas

- **Overfitting:** Propensos a sobreajustar si no se controlan (poda, profundidad, etc.).
- **Inestabilidad:** Pequeños cambios en datos pueden cambiar el árbol drásticamente.

Soluciones: - Regularización (poda, profundidad máxima). - Métodos Ensemble (Random Forest, Boosting) para reducir la inestabilidad.

Ventajas de Regresión: Simplicidad y Linealidad

i Ventajas Clave

- **Simplicidad y Eficiencia:** Modelos rápidos y eficientes, incluso con grandes datasets.
- **Relaciones Lineales:** Óptimos si la relación es lineal o aproximadamente lineal.
- **Bien Establecidos y Entendidos:** Base estadística sólida y bien estudiada.
- **Coefficientes Interpretables:** Información directa sobre variables predictoras (lineal y logística).

Ideal cuando: - Se esperan relaciones lineales. - Se busca un modelo simple y eficiente.

Desventajas de Regresión: Linealidad y Preprocesamiento

i Desventajas

- **Supuesto de Linealidad:** Rendimiento pobre si la relación es no lineal.
- **Sensibles a Outliers:** Valores atípicos pueden distorsionar el modelo.
- **Preprocesamiento Datos Categóricos:** Necesitan codificación manual.

Conclusión: Probar y Comparar

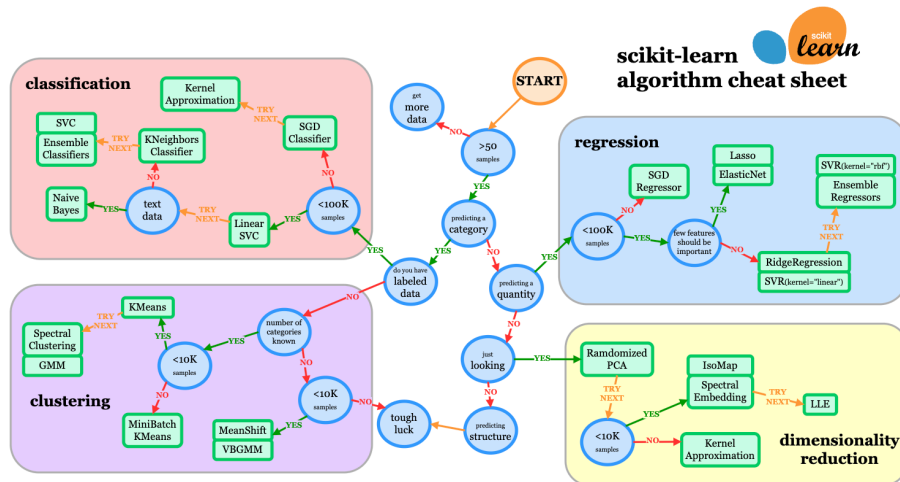
No hay “mejor” modelo universal.

Recomendación: - Probar ambos tipos de modelos: Árboles y Regresión. - Comparar el rendimiento en datos de validación. - Elegir el modelo que mejor se adapte al problema específico y los datos.

Librerías

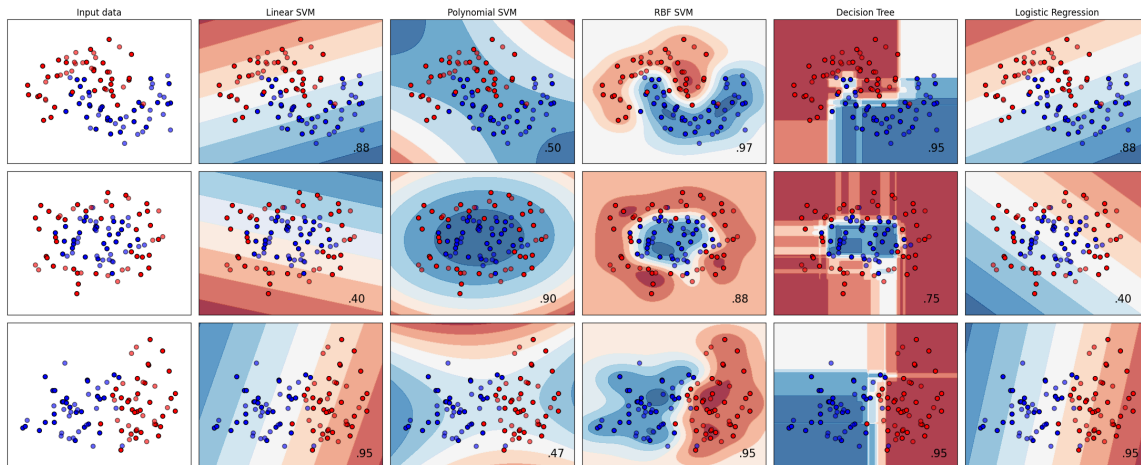
Seleccionando Algoritmos

¿Cómo elijo el algoritmo más adecuado para mis datos?



Comparación de Algoritmos

Comparación de distintos algoritmos a la fecha



Decision Tree Classifier en scikit-learn: API

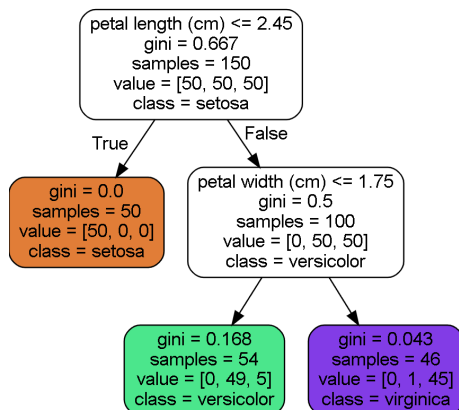
- **Clase principal:** `sklearn.tree.DecisionTreeClassifier`
- **Parámetros clave:**
 - `criterion`: La función para medir la calidad de una división ('gini' o 'entropy').
 - `max_depth`: La profundidad máxima del árbol para controlar el sobreajuste.
 - `min_samples_split`: El número mínimo de muestras requeridas para dividir un nodo.
 - `min_samples_leaf`: El número mínimo de muestras que debe tener un nodo hoja.
- **Métodos importantes:**
 - `fit(X, y)`: Entrena el modelo con datos y etiquetas.
 - `predict(X)`: Predice etiquetas para nuevos datos.
 - `score(X, y)`: Evalúa el rendimiento del modelo (precisión).

Ejemplo de Código: Árbol de Clasificación (scikit-learn)

Ejemplo de Código: Árbol de Clasificación (scikit-learn)

Clasificador Iris Dataset

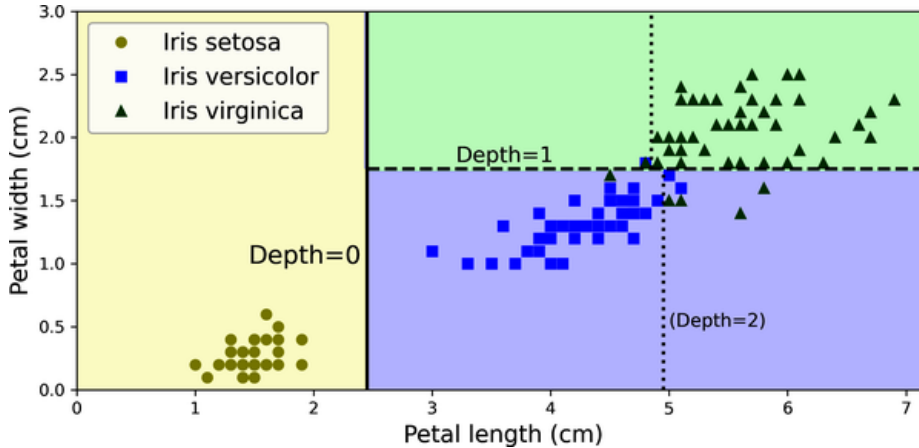
% Espacio opcional entre el texto y la imagen



Ejemplo de Código: Árbol de Clasificación (scikit-learn)

Clasificador Iris Dataset

% Espacio opcional entre el texto y la imagen



Decision Tree Regressor en scikit-learn: API

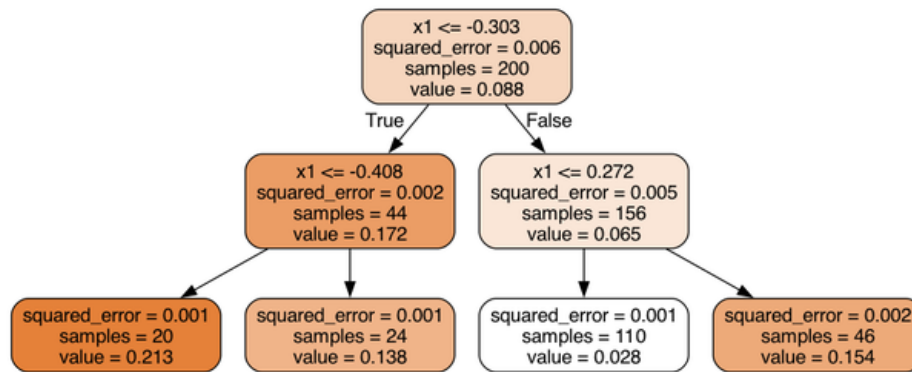
- **Clase principal:** `sklearn.tree.DecisionTreeRegressor`
- **Parámetros clave:**
 - `criterion`: La función para medir el error en una división ('squared_error', 'absolute_error').
 - `max\_depth`: La profundidad máxima del árbol.
 - `min\_samples\_split`: El número mínimo de muestras necesarias para dividir un nodo.
 - `min\_samples\_leaf`: El número mínimo de muestras en un nodo hoja.
- **Métodos importantes:**
 - `fit(X, y)`: Entrena el modelo con datos numéricos y valores numéricos.
 - `predict(X)`: Predice un valor continuo para nuevos datos.
 - `score(X, y)`: Evalúa el rendimiento mediante el coeficiente de determinación R^2 .

Ejemplo de Código: Árbol de Regresión (scikit-learn)

Ejemplo de Código: Árbol de Regresión (scikit-learn)

Aproximador a función cuadrática

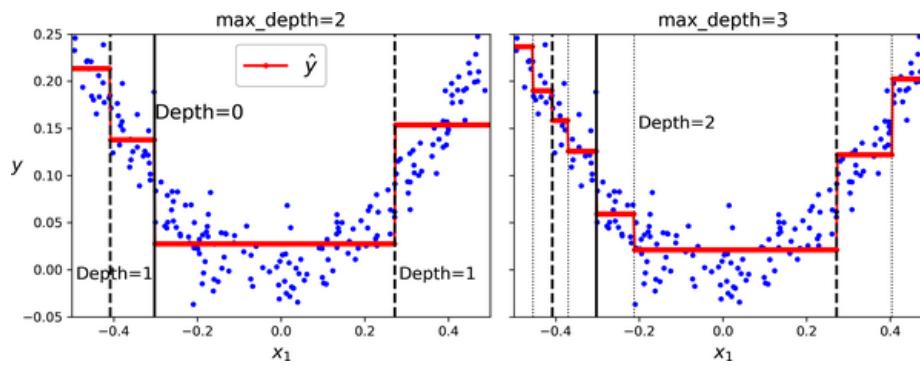
% Espacio opcional entre el texto y la imagen



Ejemplo de Código: Árbol de Regresión (scikit-learn)

Aproximador a función cuadrática

% Espacio opcional entre el texto y la imagen



References

Bibliography